

Graduate Research Assistant — Code Challenge

Build a Super Mario Bros-style platformer and train an agent to play it with Deep Reinforcement Learning (PPO).

Code & Sorcery Lab | Ontario Tech University

Bamgbose Akinlade Hussein

B.Tech. Computer Science | Bells University of Technology, Nigeria

Email: bamgboseakinlade2004@gmail.com | Phone: +2347036631786

Project context: Reinforcement Learning, Deep Reinforcement Learning and language choice

Reinforcement Learning (RL) is a type of machine learning in which an agent learns by interacting with an environment, taking actions and receiving rewards or penalties. Deep Reinforcement Learning (Deep RL) combines reinforcement learning with deep neural networks. In this project, Proximal Policy Optimization (PPO) is the reinforcement-learning algorithm, while multilayer perceptron (MLP) neural networks represent the policy (what action to take) and value function (how promising the current situation is). Unlike supervised deep learning, which usually learns from labelled examples, reinforcement learning learns from experience and reward signals. I had previous exposure to deep learning through my final-year project, and this challenge let me apply neural networks in a reinforcement-learning setting.

I used Python because the RL ecosystem needed for this challenge is strongest and easiest to integrate in Python: Pygame, Gymnasium, Stable-Baselines3, PyTorch, NumPy and Matplotlib work together directly. I understand the syntax of languages such as C# and JavaScript, but using them would have added integration work without improving the research question. Python therefore let me spend more time on the environment, PPO formulation, evaluation and analysis.

1. Problem setup and Markov Decision Process (MDP) formulation

I built one small Mario-style platformer level in Pygame and exposed the same game logic through a Gymnasium environment. The level has a fixed start and goal, platforms, two pits, a moving enemy and a fixed hazard. The player can run and jump under gravity and dies by falling out of the level or colliding with an enemy or hazard. The level itself is deterministic: resetting it gives the same layout, and the same action sequence produces the same trajectory. Headless mode lets training run without opening a Pygame window.

I used an 11-value feature vector instead of screen pixels. This keeps the experiment focused on control rather than also requiring the agent to learn visual recognition. The observation contains player position, movement information, an on-ground flag, enemy position, hazard position and goal position.

Markov Decision Process component	Definition	Why
Observation	11-dimensional float vector.	Compact and efficient for this fixed level.
Action	6 actions: no-op, left, right, jump, left+jump, right+jump.	Small action space while allowing movement and jumping together.
Reward	0.01 x horizontal progress, +10 goal, -10 death.	Progress gives feedback before the goal is discovered.
Termination	Goal reached or player dies.	Natural end of an episode.
Truncation	1,000 environment steps.	Stops an agent that remains alive without finishing.

The progress reward helps learning, but it also creates a weakness: an agent can earn some positive reward by moving right even if it eventually fails. This became important when later checkpoints made partial progress but no longer completed the level.

2. Proximal Policy Optimization (PPO) method and training procedure

I used Stable-Baselines3's Proximal Policy Optimization (PPO) implementation with its default multilayer perceptron (MLP) policy because my observations are numerical vectors. PPO uses a policy network to choose between the six possible actions and a value network to estimate the expected future reward from the current situation. I did not design a custom neural network; I used Stable-Baselines3's default architecture, which has two hidden layers of 64 units for each network.

Before the full run, I used a short smoke test to check the complete pipeline: reset the environment, select actions, calculate rewards, update PPO, evaluate checkpoints and reload a saved model. At about 20,480 training steps the agent reached x=733. It had not solved the level, but it was already much farther than the random baseline, showing that PPO was learning useful behavior from the formulation.

2. Proximal Policy Optimization (PPO) method and training procedure (continued)

Hyperparameter	Value	Meaning
Random seed	42	Makes training randomness repeatable. The value 42 is arbitrary; using the same seed supports reproducibility.
Total training budget	204,800 timesteps	Total number of environment interactions in the full training run.
Evaluation interval	10,240 timesteps	How often the current policy is tested: every 10 PPO rollout cycles.
Episodes per checkpoint evaluation	20	Number of complete attempts used to judge each checkpoint.
Learning rate	3e-4	Size of neural-network weight updates. Too large can be unstable; too small can learn slowly.
n_steps	1,024	Environment interactions collected before each PPO update.
Batch size	64	Number of collected experiences processed together during optimization.
Epochs per update	10	Number of passes PPO makes over the collected rollout data per update.
Gamma	0.99	Discount factor. At 0.99, future rewards still matter strongly.
GAE lambda	0.95	Generalized Advantage Estimation (GAE) setting that balances short- and longer-term information when estimating an action's advantage.
Clip range	0.2	Limits how far the policy can change in one PPO update, improving stability.
Entropy coefficient	0.01	Small exploration incentive that prevents the policy becoming too certain too early.

Random seed 42: I used a fixed seed of 42 to make randomness in the PPO training process repeatable, which helps make the experiment reproducible. The number 42 itself is arbitrary; what matters is keeping the seed fixed.

Evaluation interval: I evaluated the agent every 10 PPO rollout cycles. Since each rollout collects 1,024 timesteps (n_steps), this gives an evaluation interval of 10,240 timesteps.

During training, I evaluated the agent every 10,240 steps and selected the checkpoint with the highest level-completion rate. I then compared that best PPO model against a random agent over 100 episodes each.

3. Results

Policy	Mean reward	Success	Mean furthest x	Mean length
Random	-5.84	0%	131.8	697.1
Best Proximal Policy Optimization (PPO) (30,720 steps)	18.63	100%	913	187

The random agent never completed the level and reached only x=131.8 on average. The best PPO checkpoint, saved at 30,720 training steps, reached x=913, completed the level and obtained a mean reward of 18.63. It finished in 187 steps, while the random policy averaged 697.1 steps without solving the task.

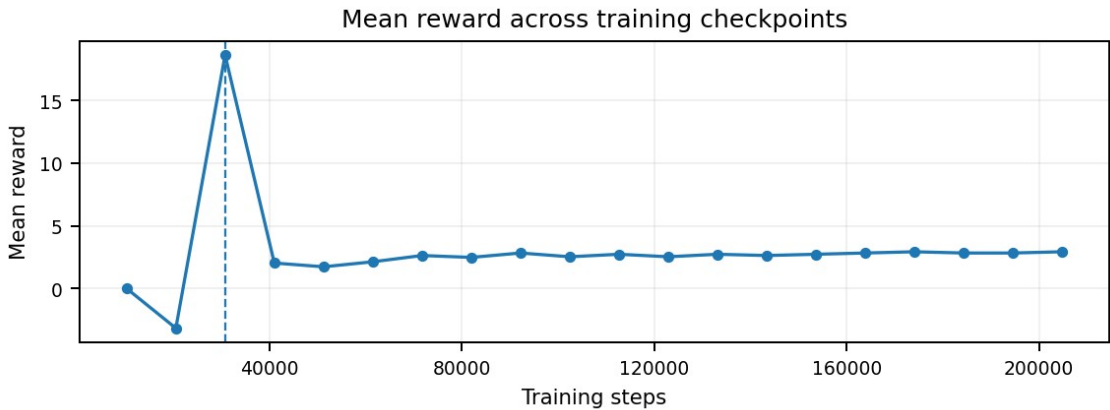


Figure 1. Mean reward across checkpoints. The dashed line marks the best checkpoint at 30,720 steps.

Figure 1 shows a sharp improvement rather than a smooth rise. Reward peaked at 18.63 when the agent solved the level at 30,720 steps, then dropped back to small positive values. This shows why the final training checkpoint would have been misleading: later policies could still earn some progress reward without completing the level.

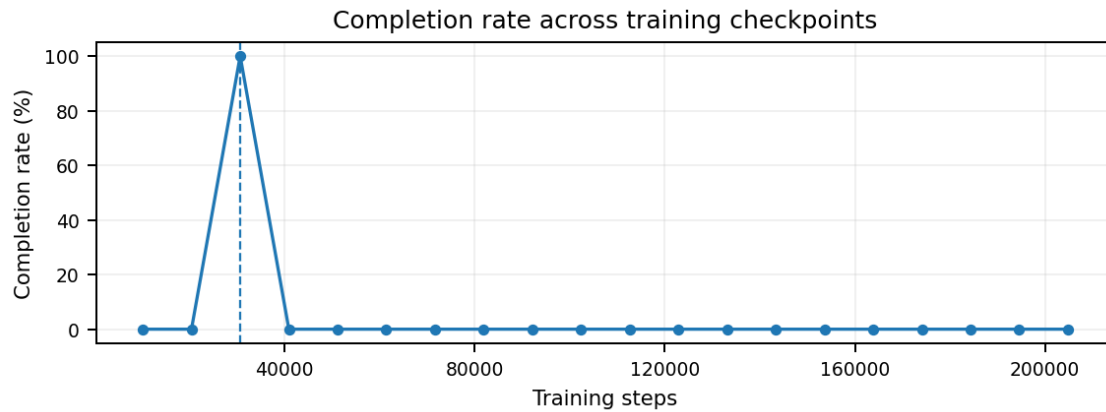


Figure 2. Completion rate across checkpoints. Only the 30,720-step checkpoint completed the fixed level.

What did not work

Training longer did not make the policy better. At 30,720 steps the policy completed the level in every deterministic checkpoint-evaluation episode. At the next checkpoint, 40,960 steps, completion fell back to 0% and did not recover during the remaining training. PPO had found a successful behavior and later moved away from it.

The shaped progress reward may have contributed to this regression because a policy can still receive positive reward for moving right without finishing. However, this experiment does not prove that reward shaping caused the collapse. It shows that PPO updates are not guaranteed to improve task-level performance continuously, and it motivates testing the reward design and update settings more carefully.

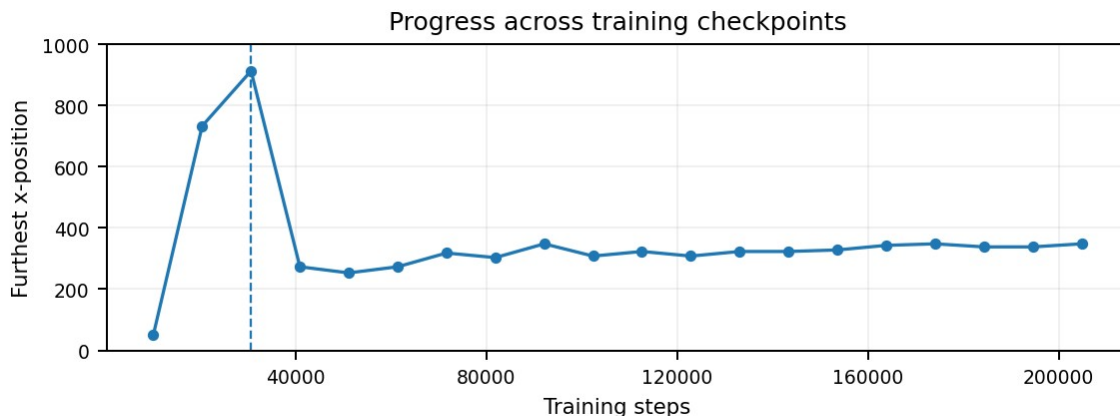


Figure 3. Furthest x-position reached. Progress peaked at the successful 30,720-step checkpoint and later regressed.

The progress curve makes the behavior easy to interpret. At 20,480 steps the agent reached $x=733$ but failed at the later hazard. At 30,720 it reached $x=913$ and won. Later checkpoints generally reached only about $x=250-350$. I kept this negative result instead of rerunning until I obtained a cleaner curve because it is useful evidence about training stability and explains why checkpointing matters.

4. Limitations and next steps

The main limitation is generalization. I trained and evaluated on one fixed game level, so the result shows that PPO learned to solve this level; it does not show that the agent can solve a new Level 2 or a different layout. The observation also contains engineered numerical features rather than raw pixels. Platform and pit locations are not explicitly included, so because the layout never changes, the player's x-position can indirectly tell the policy where obstacles are likely to be.

The evaluation is also deterministic. The selected PPO policy was run for 100 episodes, but under the same fixed conditions those episodes repeat the same trajectory. Therefore, 100% completion means the selected policy reliably solves this fixed level; it does not demonstrate robustness to new starting positions, physics noise, enemy behavior or unseen layouts.

With more time, I would first run a reward-shaping ablation: compare the current progress reward with a sparse goal/death reward and a version with a small time penalty. An ablation changes one design choice at a time so its effect can be measured. I would also test a lower learning rate and different entropy coefficients to investigate why performance collapsed after 30,720 steps.

Next, I would randomize starting positions, enemy locations and level layouts. The game code would define the allowed variation, while different random seeds would produce reproducible random choices within those rules. I would train on some layouts and evaluate on held-out layouts that the agent had never seen. Finally, I would repeat each configuration across multiple seeds and report the variation or confidence intervals, rather than relying on one training run.

5. Reproducibility, code structure and disclosure

The repository contains standalone training and evaluation scripts, a JavaScript Object Notation (JSON) configuration file, pinned dependencies, saved model weights and machine-readable results. The standalone run reproduced the same best checkpoint at 30,720 steps and the same headline evaluation. Training for 204,800 timesteps took about 52.3 seconds on a MacBook Pro with an Apple M5 processor (10 cores) and 16 GB RAM using Python 3.13.15. I also created a fresh virtual environment from requirements.txt and successfully loaded and evaluated the saved model.

Code structure

File	Purpose
sprites.py	Defines the player, platforms, enemy, hazard and goal sprites.
game.py	Contains the level, physics, collisions, actions, reset logic and win/death rules.
main.py	Runs the game for manual keyboard play.
environment.py	Wraps the game as a Gymnasium environment with reset(), step(), observations and rewards.
train.py	Creates the Proximal Policy Optimization (PPO) agent, trains in rollout chunks, evaluates checkpoints and saves the best model/results.
evaluate.py	Loads and compares the saved PPO model with the random baseline; can also render playback.
config.json	Stores experiment settings and PPO hyperparameters in JavaScript Object Notation (JSON) format for easy reproduction and changes.
test_environment.py	Checks the custom environment, including reset/step behavior and determinism.
notebooks/ppo_experiments.ipynb	Used for exploratory PPO experiments and early analysis.

Libraries used were Pygame for the game/rendering, Gymnasium for the reinforcement-learning environment application programming interface (API), Stable-Baselines3 for PPO, PyTorch underneath Stable-Baselines3 for neural-network training, NumPy for numerical data, Matplotlib for plots and JupyterLab for exploratory work. I did not use an existing Mario RL environment or a pretrained agent.

Tutorials and learning resources

- 1. Clear Code - The ultimate introduction to Pygame: [YouTube tutorial](#)
- 2. Johnny Code - Custom Gymnasium environment and Stable-Baselines3: [YouTube tutorial](#)
- 3. Additional reinforcement-learning / Proximal Policy Optimization (PPO) learning resource: [YouTube tutorial](#)

I used these tutorials as learning references for Pygame, Gymnasium and reinforcement-learning concepts rather than copying an existing Mario environment.

AI assistance and verification

I used ChatGPT as a development and learning assistant to explain unfamiliar concepts and code, discuss the game and Markov Decision Process (MDP) design, debug implementation issues, structure PPO experiments and reproducibility checks, and help edit the README and write-up. I ran and checked the project locally throughout: manual gameplay, Gymnasium environment validation, deterministic trajectory tests, PPO training, random-baseline comparison, saved-model loading, fresh-environment evaluation and rendered agent playback. I kept the implementation small enough that I can explain the code and design decisions directly.